COMP100-Spring2026-PS5

The test cases given are just sample tests, and additional tests may be used while grading.

Description

You will design a base Drone class and two specialized subclasses (DeliveryDrone and SurveyDrone), then compose them into a DroneHub that manages these drones collectively.

Task Overview

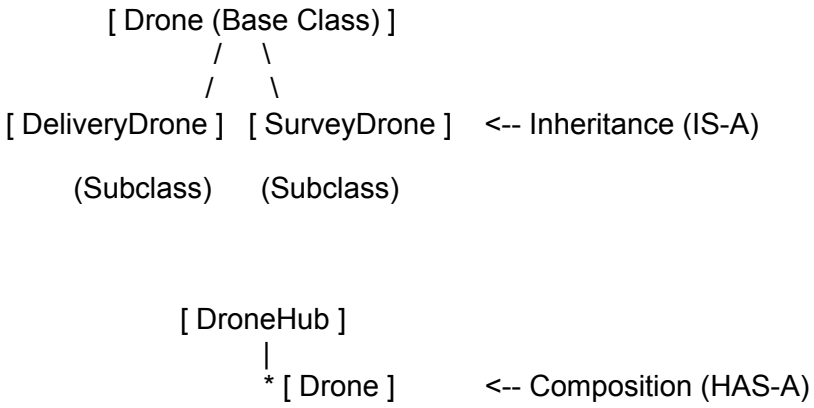
You will:

1. Implement a Drone base class (Task 1).
2. Implement two subclasses, DeliveryDrone and SurveyDrone (Task 2).
3. Implement a DroneHub class for fleet management (Task 3).
4. Implement custom exception classes and refactor your drones for robust error handling (Task 4).

How to Approach This Assignment

1. **Start with drone.py:** This is your foundation. Focus on the class variable ids and basic initialization. For now, you can use a simple *print* or *assert* to check for duplicate IDs.
2. **Move to Subclasses:** Implement DeliveryDrone and SurveyDrone. Remember to use super() to reuse the code you already wrote in the base class.
3. **Implement DroneHub:** This class uses Composition. It doesn't inherit from anything, but it contains a list of drones and manages them.
4. **Finish with drone_exceptions.py:** Define your custom error types here and update your drone classes to raise these exceptions instead of just printing errors.
5. **Test Frequently:** Run *python simulator.py* or use the provided test files (e.g., *python drone_test.py*) to check your progress.

Conceptual Model



Understanding Classes, Inheritance, and Subclasses

What is a Class?

A **class** is a blueprint or template that defines what attributes (data) and methods (functions) objects of that type should have. For example:

```python
class Vehicle:
    def __init__(self, color):
        self.color = color

    def move(self):
        print("The vehicle is moving.")
```

Here, `Vehicle` is a class. Every `Vehicle` has a `color` and a `move()` method.

What is an Object?

When you create a specific instance of a class, it's called an **object**. For example:

```python
my_vehicle = Vehicle("red")
my_vehicle.move()  # Prints: "The vehicle is moving."
```

my_vehicle is now an object of type Vehicle with its own color and the ability to move().

What is Inheritance?

Inheritance allows you to create a new class that builds upon an existing class. The new class (a **subclass**) gets all the features of the existing class (the **base class**) and can add or modify them.

What is a Subclass?

A **subclass** is a class that inherits from another class. For example, Car might be a subclass of Vehicle:

```python
class Car(Vehicle):
    def __init__(self, color, make, model):
        super().__init__(color)  # Call the parent constructor
        self.make = make
        self.model = model

    def move(self):
        # Override the move method from Vehicle
        print(f"The {self.color} {self.make} {self.model} is driving on the
road.")
```

Here's what is happening:

- Car inherits from Vehicle, so it starts with what Vehicle provides.
- By calling super().__init__(color), we ensure that the parent class (Vehicle) sets up the basic attributes like color.
- We add new attributes make and model to Car.

- We override the move() method to provide a more specific message for cars.

When we create a Car object:

```python
my_car = Car("blue", "Honda", "Civic")
my_car.move()  # Prints: "The blue Honda Civic is driving on the road."
```

This shows that Car is a specialized version of Vehicle.

Applying These Concepts to Our Drone Scenario

In this problem set:

- We start with a Drone class that defines general features and behaviors shared by all drones.
- We then create subclasses DeliveryDrone and SurveyDrone that inherit from Drone. Because of inheritance, these subclasses automatically have all the features of Drone, and we can add or override features to make them unique.
- Finally, we use a DroneHub class to manage multiple drones (of various types) together.

Detailed Requirements

1. Drone (Base Class)

- **Attributes**:
  - id (string): Unique identifier for the drone.
  - ids (class variable, list): Shared across all drones. Initially an empty list. It stores all existing drone IDs. Every new drone must append its ID to this list.
  - max_speed (int): Maximum speed (km/h).
  - current_location (tuple, e.g. (x, y) or (lat, lon)): Current position.
- **Methods**:
  - __init__(self, id, max_speed, current_location): Initialize with given attributes.
    - Check if the id is already in the ids list. If it is, raise a DuplicateIDError (from drone_exceptions.py).
    - After successful validation, add id to the class variable ids.

- move_to(self, new_location): Update current_location to new_location.
- get_type(self): Return "Generic Drone" by default.

This sets the foundation for all drones.

2. DeliveryDrone (Subclass of Drone)

- **Additional Attributes**:
  - cargo_capacity (float): Maximum cargo weight.
  - current_cargo (float): Current cargo load (starts at 0).
- **Methods**:
  - __init__(self, id, max_speed, current_location, cargo_capacity): Calls super() and sets cargo_capacity.
  - load_cargo(self, weight): Adds weight to current_cargo if it does not exceed cargo_capacity. Otherwise, raise a CargoLimitError (from drone_exceptions.py).
  - deliver_cargo(self): Sets current_cargo to 0.0 and prints a delivery success message: "Cargo delivered successfully!".
  - get_type(self): Override to return "Delivery Drone".

3. SurveyDrone (Subclass of Drone)

- **Additional Attributes**:
  - camera_quality (string): e.g., "4K", "HD".
  - survey_data (list): Stores information about scanned areas.
- **Methods**:
  - __init__(self, id, max_speed, current_location, camera_quality): Calls super() and sets camera_quality. Initializes survey_data as empty list.
  - scan_area(self, area_coordinates): Simulates scanning the given area (a list of coordinates). Adds all coordinates from the list to survey_data (extending the data) and prints a completion message: "Survey completed!".
  - get_type(self): Override to return "Survey Drone".

4. DroneHub

- **Attributes**:
  - name (string): Name of the drone hub.
  - drones (list): A list of Drone objects (including DeliveryDrone and SurveyDrone).
- **Methods**:
  - __init__(self, name): Sets the hub name and initializes an empty drones list.
  - add_drone(self, drone): Adds a Drone object to drones.
  - list_drones(self): Prints each drone's ID and its type (by calling get_type()).

- relocate_all_drones(self, new_location): Calls move_to(new_location) on each drone.
- __str__(self): When a DroneHub object is printed, it displays the hub name and a list of drones docked. **Note:** The returned string must start with a newline character (\n).

5. drone_exceptions.py (Custom Exceptions)

You must implement the following exception classes to handle errors gracefully:

- DroneError: A base exception class that inherits from Python's built-in Exception.
- DuplicateIDError: Inherits from DroneError. It should accept a drone_id and initialize its message as: "Drone id {drone_id} already exists!".
- CargoLimitError: Inherits from DroneError. It should accept an optional message (default: "Weight exceeds maximum capacity!").

Expected Simulator Output

When you run *python simulator.py* after completing all tasks, your output should look like this:

```
Shell
--- Phase 1: Creating Drones ---
Created Delivery Drone with ID #D1001
Created Survey Drone with ID #S1001

--- Phase 2: Specific Drone Actions ---
Loading 10kg into Delivery Drone...
Delivering cargo...
Cargo delivered successfully!

Scanning area with Survey Drone...
Survey completed!

--- Phase 3: Handling Errors ---
Trying to load 50kg into drone with capacity 45.0...
Caught expected error: Weight exceeds maximum capacity!

Trying to create a drone with an existing ID '#D1001'...
Caught expected error: Drone id #D1001 already exists!

--- Phase 4: Hub Management ---
Listing all drones in hub 'Central Fleet':
Drone ID: #D1001 Type: Delivery Drone
```

```
Drone ID: #S1001 Type: Survey Drone
Drone ID: #D1002 Type: Delivery Drone

Testing Hub __str__:

--- DroneHub Central Fleet: 3 drones docked ---
[0]: #D1001 Delivery Drone
[1]: #S1001 Survey Drone
[2]: #D1002 Delivery Drone

--- Phase 5: Bulk Relocation ---
Relocating all drones to (50.0, -120.0)...

Verifying location of first drone...
Drone #D1001 location: (50.0, -120.0)
```